



```
import A5O
import Closure
set_option autoImplicit false
```

```
/-! # A5O — Hops: one gate for single and multiple hops.
Companion to A5O.lean v4 (SHA-256 0e5ebe3f...5b452) and Closure.lean
(SHA-256 af6b3233...a933), both imported unmodified.
```

Claim under test (Chairman, 2026-09-17): a hop is not a new object. An appointed agent that appoints is, at that moment, a principal, and that appointment is decided by the SAME gate against the SAME ledger. Multi-hop is single-hop iterated; revocation at any link defeats every link below it; the trace terminates at one root.

Modelling choices, stated:

- * One type `Party` serves as v4's `Principal` and `Agent`, because an appointee may appoint.
 - * `Time := Nat` so termination is by induction on time, with no axiom.
 - * `appointAct A X : Action` is the act "appoint A for X". The ledger records it as an execution of the appointer (a grant is written by someone). It is in an area of consequence whenever X is.
 - * `Root P` marks a party that is not itself appointed (the operator).
- /

namespace Hops

```
variable {Party Action AoC : Type}
variable (C : Party → Party → Action → Nat → Prop)
variable (I : Party → Prop)
variable (M : Party → Action → Prop)
variable (S : Party → Action → Prop)
variable (R : Party → Party → Action → Nat → Prop)
variable (Appointed : Party → Party → Action → Nat → Prop)
variable (Revoke : Party → Party → Action → Nat → Prop)
variable (Executes : Party → Action → Nat → Prop)
variable (inAoC : Action → AoC → Prop)
variable (appointAct : Party → Action → Action)
variable (Root : Party → Prop)
```

-- ---- Ledger semantics (the gate's rules, as hypotheses) -----

```
/-- H1. Appointing in an area of consequence is itself in that area. -/
def AppointIsConsequential : Prop :=
  ∀ (A : Party) (X : Action) (d : AoC), inAoC X d → inAoC (appointAct A X) d
```

-- H2. Origin: every appointment of A for X at t was either made by a root, or was itself an execution of the appointing act by a non-root party at an earlier time $s < t$ (a grant is written before it is decided against). -/

def Origin : Prop :=

$\forall (P A : \text{Party}) (X : \text{Action}) (t : \text{Nat}), \text{Appointed } P A X t \rightarrow$
 $\text{Root } P \vee \exists s : \text{Nat}, s < t \wedge \text{Executes } P (\text{appointAct } A X) s$

-- H3. Resolution: revocation-dominance for a non-root apppointer P at t holds only if P's own appointment for the appointing act still stands at t (the gate resolves `appointment\_ref` as of the moment before execution). -/

def Resolves : Prop :=

$\forall (P A : \text{Party}) (X : \text{Action}) (t : \text{Nat}), R P A X t \rightarrow$
 $\text{Root } P \vee \exists (Q : \text{Party}) (s : \text{Nat}), s < t \wedge$
 $\text{Appointed } Q P (\text{appointAct } A X) s \wedge R Q P (\text{appointAct } A X) t$

-- H4. One grant per (appointee, act): the ledger's `appointment\_ref` resolves to one apppointer. -/

def OneApppointer : Prop :=

$\forall (Q Q' P : \text{Party}) (Y : \text{Action}) (s s' : \text{Nat}),$
 $\text{Appointed } Q P Y s \rightarrow \text{Appointed } Q' P Y s' \rightarrow Q = Q'$

-- H5. Revocation is append-only: once revoked, revoked. -/

def RevokePersists : Prop :=

$\forall (Q P : \text{Party}) (Y : \text{Action}) (t t' : \text{Nat}),$
 $t \leq t' \rightarrow \text{Revoke } Q P Y t \rightarrow \text{Revoke } Q P Y t'$

-- v4's P2 shape, as consumed by `revoked\_defeats\_O`. -/

def P2 : Prop :=

$\forall (P A : \text{Party}) (X : \text{Action}) (t t' : \text{Nat}),$
 $t < t' \rightarrow \text{Appointed } P A X t \rightarrow \text{Revoke } P A X t' \rightarrow \neg R P A X t'$

-- ---- The trace -----

-- `Traces A X t`: A's authority for X at t resolves, through O-links only, to a root. `hop` is the multi-hop case; note it is built from exactly the same O-instance as `root` — one decision shape at every level. -/

inductive Traces : Party $\rightarrow$ Action $\rightarrow$ Nat $\rightarrow$ Prop

| root (P A : Party) (X : Action) (t : Nat) :

$\text{Root } P \rightarrow \text{O C I M S R Appointed } P A X t \rightarrow \text{Traces } A X t$

| hop (P A : Party) (X : Action) (s t : Nat) :

$s < t \rightarrow \text{O C I M S R Appointed } P A X t \rightarrow$

$\text{Traces } P (\text{appointAct } A X) s \rightarrow \text{Traces } A X t$

-- ---- Theorem 1: every gated execution traces to a root -----

/-- Strong induction on Nat, derived from ordinary induction (no axiom). -/

theorem strong_ind (motive : Nat → Prop)

(h : ∀ n, (∀ m, m < n → motive m) → motive n) : ∀ n, motive n := by

intro n

have key : ∀ k, ∀ m, m < k → motive m := by

intro k

induction k with

| zero => intro m hm; exact absurd hm (Nat.not_lt_zero m)

| succ k ih =>

intro m hm

cases Nat.lt_succ_iff_lt_or_eq.mp hm with

| inl hlt => exact ih m hlt

| inr heq => subst heq; exact h m ih

exact key (n+1) n (Nat.lt_succ_self n)

/-- ONE GATE, ANY DEPTH. If the gate is in the execution path (Closure's
`GatedSystem`), the system achieves v4's ADA, and the ledger obeys H1–H2,
then every execution in an area of consequence — at any hop depth —
traces through O-links to a root. -/

theorem every_execution_traces

(G : GatedSystem Executes inAoC)

(hADA : achievesADA C I M S R Appointed inAoC G.sys)

(h1 : AppointIsConsequential inAoC appointAct)

(h2 : Origin Appointed Executes appointAct Root) :

∀ (t : Nat) (A : Party) (X : Action) (d : AoC),

inAoC X d → Executes A X t →

Traces C I M S R Appointed appointAct Root A X t := by

refine strong_ind (fun t => ∀ (A : Party) (X : Action) (d : AoC),

inAoC X d → Executes A X t →

Traces C I M S R Appointed appointAct Root A X t) ?_

intro t ih A X d hd hExec

obtain ⟨P, hEnf⟩ := G.intercepts A X t d hd hExec

have hO : O C I M S R Appointed P A X t := hADA P A X t d hd hEnf

have hApp : Appointed P A X t := hO.2.2.2.2

cases h2 P A X t hApp with

| inl hRoot => exact Traces.root P A X t hRoot hO

| inr hEx =>

obtain ⟨s, hs, hExecP⟩ := hEx

have hTr : Traces C I M S R Appointed appointAct Root P (appointAct A X) s :=

ih s hs P (appointAct A X) d (h1 A X d hd) hExecP

exact Traces.hop P A X s t hs hO hTr

-- The top of every trace is a root. --

```
theorem trace_has_root (A : Party) (X : Action) (t : Nat)
  (h : Traces C I M S R Appointed appointAct Root A X t) :
  ∃ P : Party, Root P := by
  induction h with
  | root P _ _ _ hR _ => exact ⟨P, hR⟩
  | hop _ _ _ _ _ _ ih => exact ih
```

-- Every link, including the last, is an O-instance: the hop case adds no condition beyond O. --

```
theorem trace_immediate (A : Party) (X : Action) (t : Nat)
  (h : Traces C I M S R Appointed appointAct Root A X t) :
  ∃ P : Party, O C I M S R Appointed P A X t := by
  cases h with
  | root P _ _ _ _ hO => exact ⟨P, hO⟩
  | hop P _ _ _ _ _ hO _ => exact ⟨P, hO⟩
```

-- ---- Theorem 2: uniqueness at every link -----

-- Under CFunctional, each link of a trace has one governor: v4's
`O\_unique`, read at every hop. The trace is a function of the execution,
not a choice. --

```
theorem link_unique (hC : CFunctional C) (P P' A : Party) (X : Action) (t : Nat)
  (h : O C I M S R Appointed P A X t) (h' : O C I M S R Appointed P' A X t) :
  P = P' :=
  O_unique C I M S R Appointed hC P P' A X t h h'
```

-- ---- Theorem 3: revocation cascades -----

-- CASCADE. If Q appointed P for the act of appointing A for X, and Q later
revokes that, then P satisfies O for nothing downstream of it afterward:
O P A X t' fails for every t' after the revocation. No cascade rule is
needed beyond the gate resolving `appointment\_ref` at decision time (H3). --

```
theorem revocation_cascades
  (hP2 : P2 R Appointed Revoke)
  (h3 : Resolves R Appointed appointAct Root)
  (h4 : OneAppointer Appointed)
  (h5 : RevokePersists Revoke)
  (P Q A : Party) (X : Action) (s t' t'' : Nat)
  (hnr : ¬ Root P)
  (hApp : Appointed Q P (appointAct A X) s)
  (hs : s < t') (hRev : Revoke Q P (appointAct A X) t') (hlt : t' < t'') :
  ¬ O C I M S R Appointed P A X t'' := by
  intro hO
```

```

have hR : R P A X t'' := hO.2.2.2.1
cases h3 P A X t'' hR with
| inl hRoot => exact hnr hRoot
| inr hEx =>
  obtain ⟨Q', s', _, hApp', hR'⟩ := hEx
  have hQ : Q = Q' := h4 Q Q' P (appointAct A X) s s' hApp hApp'
  subst hQ
  have hRev'' : Revoke Q P (appointAct A X) t'' :=
    h5 Q P (appointAct A X) t' t'' (Nat.le_of_lt hlt) hRev
  have hst : s < t'' := Nat.lt_trans hs hlt
  exact hP2 Q P (appointAct A X) s t'' hst hApp hRev'' hR'

```

-- Operational form: after Q revokes P upstream, a gated ADA system will not enforce A's execution of X for P at any later time. Same shape as v4's `revocation\_blocks\_enforcement`, one hop up. -/

```

theorem cascade_blocks_enforcement
  (hP2 : P2 R Appointed Revoke)
  (h3 : Resolves R Appointed appointAct Root)
  (h4 : OneAppointer Appointed)
  (h5 : RevokePersists Revoke)
  (G : GatedSystem Executes inAoC)
  (hADA : achievesADA C I M S R Appointed inAoC G.sys)
  (P Q A : Party) (X : Action) (s t' t'' : Nat) (d : AoC)
  (hd : inAoC X d) (hnr : ¬ Root P)
  (hApp : Appointed Q P (appointAct A X) s)
  (hs : s < t') (hRev : Revoke Q P (appointAct A X) t') (hlt : t' < t'') :
  ¬ G.sys.enforces P A X t'' :=
fun hEnf =>
  revocation_cascades C I M S R Appointed Revoke appointAct Root
  hP2 h3 h4 h5 P Q A X s t' t'' hnr hApp hs hRev hlt
  (hADA P A X t'' d hd hEnf)

```

end Hops

```

#print axioms Hops.strong_ind
#print axioms Hops.every_execution_traces
#print axioms Hops.trace_has_root
#print axioms Hops.trace_immediate
#print axioms Hops.link_unique
#print axioms Hops.revocation_cascades
#print axioms Hops.cascade_blocks_enforcement

```